Agentic AI in Pega: The Guide Everyone Wanted But No One Bothered to Write

A Practical, Funny, No-Nonsense Guide to Gen AI Agents, Tools, Case Types & Real Scenario

This document serves as a starter guide that helps you complete an end-to-end use case on dealing with AI Agents be it a single agent or multi agents the fundamentals are same.

In this document we are going to outline about the 5 patterns of using Agentic AI rules in PEGA and how the configuration of rules play an important role. But wait, before we go into knowing the patterns on how to use them first you need to understand the basic nitty gritty of the rule and the significance of the options.

1. Agent
2. Tool
3. Gen AI Document

1. What Is a Gen AI Agent?

A **Gen AI Agent** in Pega is:

A conversational LLM-powered assistant that can reason, ask questions, call tools, create cases, update data, and guide users through workflows.

It is NOT:

- A chatbot
- A wrapper around ChatGPT
- A fancy UI gimmick

It is a **reasoning engine** that:

- Understands intent of the user
- Collects missing information
- Calls Pega rules (Tools)
- Creates or updates cases
- Summarizes results
- Follows guardrails

All **without the user navigating screens, forms, or menus**. This is why embedding an Agent inside a case or portal is powerful.

Think of it as:

A junior developer who can talk to users, ask questions, and run rules — but only the rules you explicitly allow.

So where to use it?

A Gen AI Agent is used inside a case or end-user portal to replace complex UI navigation with conversational, intent-driven case processing. It allows users to perform actions, update data, create cases, and move workflows forward without needing to understand the systems structure. This reduces training, improves accessibility, accelerates case completion, and enables dynamic orchestration of tools and case actions.

Let's navigate each section with a simple scenario:

Consider Clinical Incident Management System where clinical team logs incidents for patients, creates and updates patients details, sometimes deletes incorrect entries and logs Clinical Incidents. So, we are going to create an Agent that achieves the functionality.

Agents in End user Work Portal can be enabled by creating a Landing Page and configuring the landing page in the Work user portal.

2. Anatomy of a Gen AI Agent Rule

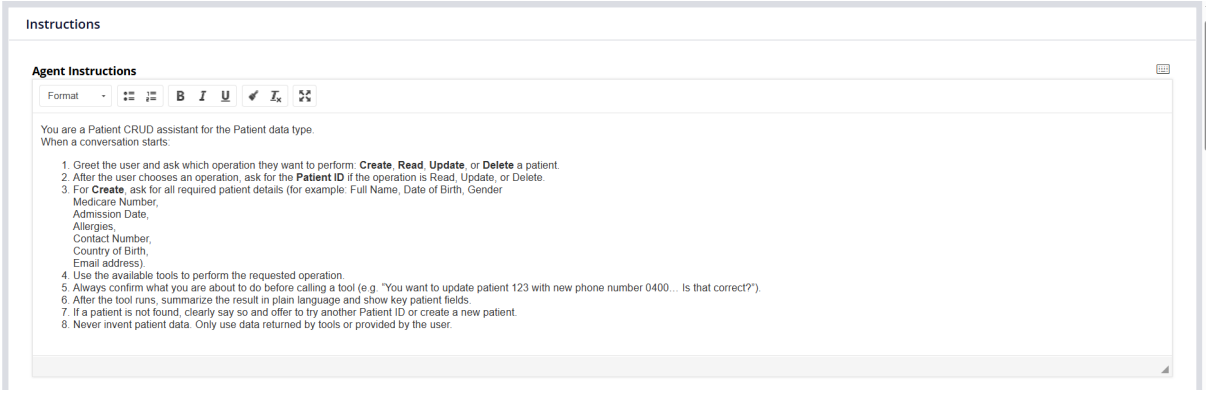
The Agent rule has several tabs. Each one has a very specific purpose.

2.1 Definition Tab

This is the **brain** of the Agent.

Contains:

- **Agent Instructions:** In this section you define the core behaviour of the agent. That's mean you tell the agent how to think not what to do about or which tool to use. These instructions should shape the Agents reasoning.



- **Response Style:** Here you define the style and tone of the response when agent responds to the users during the conversation whether its presenting results providing response etc.

Eg: Professional, concise, and clear. Use bullet points when summarizing patient details and providing valid response to the user related to patients CRUD operations.

- **Guardrails:** Guardrails define the Agent's limits, safety rules, and forbidden behaviours. It ensures agent stays complaint, safe and predictable.
    - No Hallucination Rules
    - Safety & Confirmation Rules
    - Data Privacy Rules
    - Scope Limitation Rules
    - Tool Usage Restrictions
    - No Assumptions about missing data
    - No unauthorization updates
    - and list goes on.

You basically draw boundaries to your agent

- **Additional Context (Data Pages):** Additional Context is for read-only Data Pages that provide background knowledge the Agent can reference during reasoning. They are loaded automatically at startup (if parameterless) and reused during the conversation. They are NOT tools and cannot perform CRUD or case actions.

  Eg: If user want to create an Incident, you can specify a data page that lists the Incident Types like Medication Error, Pressure Injury, Equipment failure etc.

- **Starter Questions:** This question is most important in the Agent configuration as this guides agent for a starting greeting message to user that talks about the capabilities of the agent so that it's easy for user to understand the capabilities of the Agent. If you don't give any Starter Question, the Agent is just an empty chat box without any greeting messages. It's always good to have a starter question else it impacts the performance of the Agent as it doesn't know the context.

  Eg: How can I help you today? You can log a new incident, update an existing one, retrieve incident details, or delete an incorrect entry, perform CRUD operations on patient record.

  The user immediately knows What the agent can do, what actions are available and how to start.

- **Quick Selection Questions.** These are the questions that users can select directly from the **Suggestions options**(looks like chat icon) in Agent window instead of prompting to agent. Can be used as part of the conversation. When the option is selected agent follows the instructions defined against the question to take next steps when the option is selected by user.

- **Additional Instructions:** For both question types you specify instructions associated with the Question. For instance, for Starter question you specify instructions on when the question should be asked to agent.

Eg:

- Immediately when the Agent opens

- Before the user types anything

- Before any tools are invoked

- **Tools (LLM-callable actions)**

We'll walk through **each Agent configuration option** and show **how it behaves** in this scenario.

Now let's navigate the rule

Example Agent Instructions (for CRUD):

You are a Patient CRUD assistant. When the conversation starts, ask the user which operation they want: Create, Read, Update, or Delete. Ask for PatientID when needed. Confirm intent before calling any tool. Never invent data. Only use data provided by the user or returned by tools. Summarize results clearly.

2.2 Knowledge Tab → Tools

This is the **toolbox** the LLM can use.

Each tool has:

- **Description** → What the tool does and what should be the response
- **Example phrases** → When the tool should be used. When LLM finds this example phrase it directly uses this tool instead of skimming other tools.
- **Parameter mapping** → What inputs it needs. The attributes that are asked as part of the conversation are considered as parameters so the name should match exactly.

Tools here are used **during reasoning**.

A Tool with category as Data Page can only be used under Tools but not in the Actions as tools under Actions tab does not support tool with category as data page. Because data pages are used specifically for data read and CRUD operations but not the actions that are triggered by user.

Supported tool types:

- Data Pages (read-only or savable)
- Integrations
- Case actions
- Automations

NOT supported:

- Activities
- Flows directly
- UI actions

2.3 Case Types Tab

This tells the Agent what case types the agent can create. Any case type can be accessed only through Tool rule. Which means for each case type a tool rule should be created with Case Type as option.

“You are allowed to create or work on these case types during the conversation.”

This is **LLM-driven case creation**.

Example:

User: “I want to report an incident.”
Agent:

- Recognizes intent
- Asks questions
- Creates the case
- Populates fields

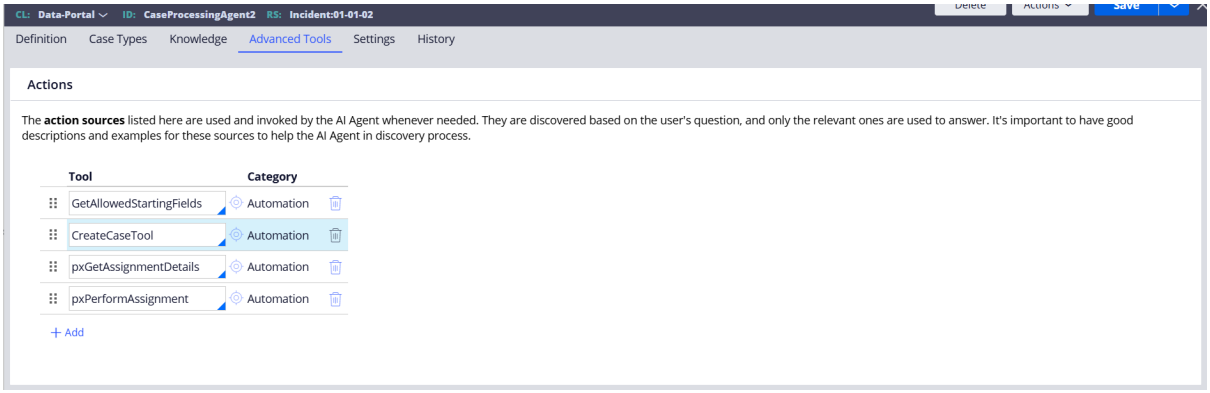
Note:

If you want to create cases from the user’s Work Portal through Landing Pages, this option becomes obsolete. In such scenarios, case creation must be done using a Tool configured under the **Automation** option and added through **Advanced Tools**. This is because the Gen AI Agent determines context based on inheritance and the class on which the Agent rule is defined. The *Applies-To* class of the Agent plays a critical role in deciding which Tools and Case Types are available.

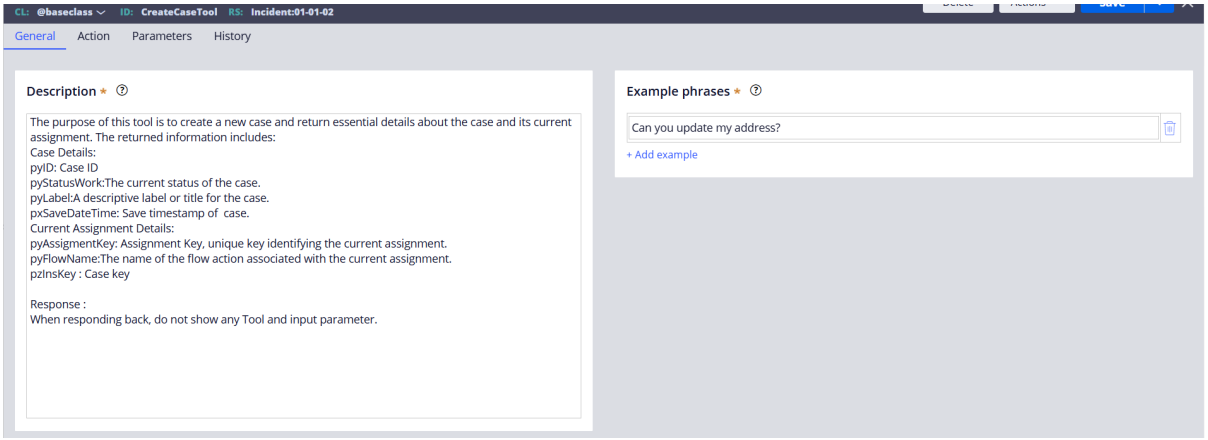
So when is this tab actually useful?

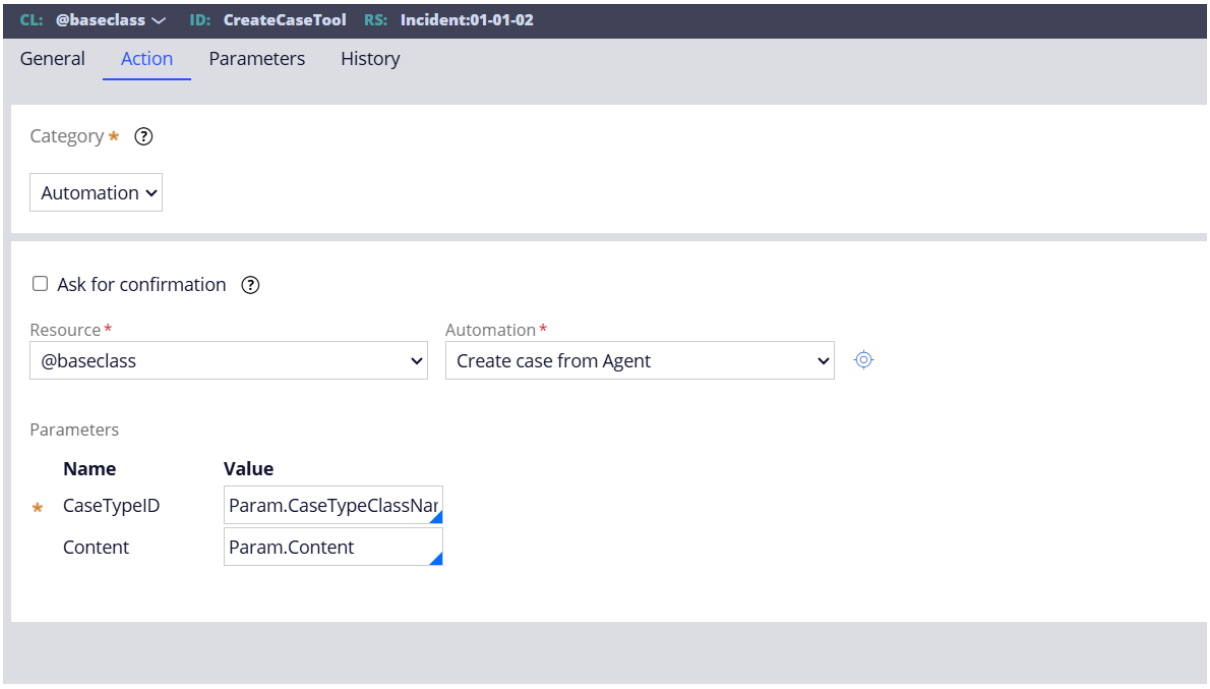
This tab is intended for **case creation within a case interaction**. For each case type, there is an option in App Studio (under the Case Type UX tab) that allows enabling the Agent for that specific case type.

Example: Creating a case from a Data-Portal Landing Page configuration



In the above screen shot CreateCaseTool is an automation tool that is internally configured with an Automation that invokes activity to create case. Based on the interaction agent identifies the case type class and interaction content are passed as input for case creation.

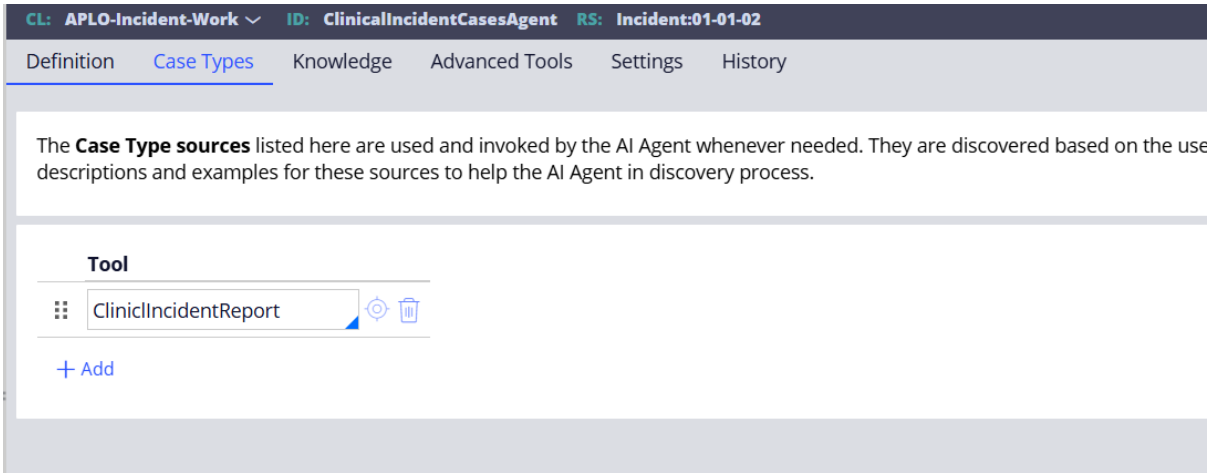


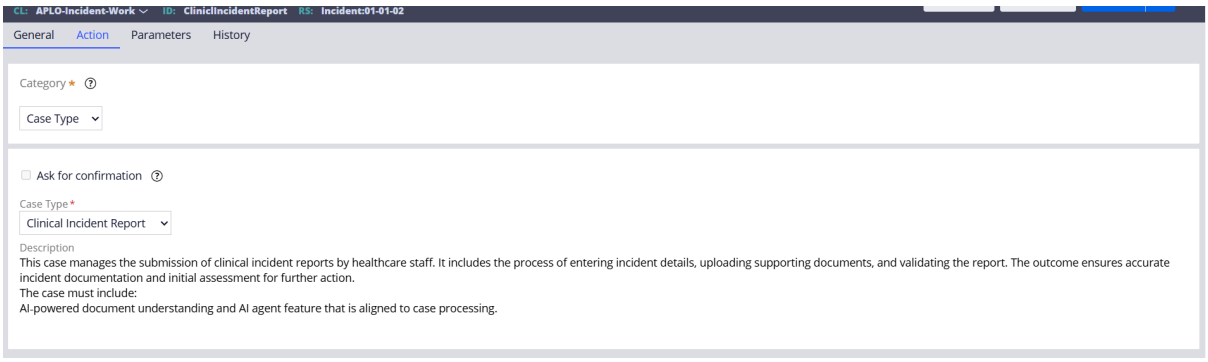


Create case from Agent is OOTB agent automation.

Example 2: Case creation from Case landing page.

If you want to create case as part of other case interaction then Case Types tab is useful. As the agent rule is created in Work class Agent automatically detects the Case Types based on user intent to create respective case.





2.4 Advanced Tools → Actions Tab

This is for **user-triggered actions**, not LLM reasoning.

Examples:

- Start a case: When creating cases from end user portal
- Run a flow: Invoke an automation flow Eg: Read data and send email, Generate PDF summaries of a data and email to user.
- Move case to next stage: Change case stage by providing case ID.

All the above tools are automations that should be written using activities. So the conversation data that is required for the automation to process should be passed as input parameters to the automation configured in the Tool else the data wont be passed as the agent doesn't know what attributes to use for which operation.

Key rule:

- Data Pages cannot be configured here
- Case actions and automations can

Settings

Under this section you can choose the AI model that AI Agent should use for interaction. Choosing a different **AI model** in the Agent's *Settings* tab **does change both the Agent's behaviour and your token usage** in terms of:

1. Response quality and reasoning depth change

More advanced models produce:

- Better intent recognition
- More accurate follow-up questions
- Fewer hallucinations

2. Token consumption increases with more capable models

Stronger models:

- Use more tokens per response
- Produce longer, more detailed answers
- Cost more per interaction

Lighter models:

- Use fewer tokens
- Give shorter, more direct answers

3. Conversation style and tone shift

4. Tool-calling accuracy improves with stronger models

To compare the difference, you can try same scenario with same inputs by configuring different AI models the difference can be seen in terms of token consumptions, context understanding etc and the response time.

3. AI Placement patterns

Here are the 4 ai placement patters that we have tried based on the provided PEGA video which I will make it easy for you to understand. One of them is repetition of assignment insights in the video.

https://community.pega.com/video-library/pega-predictable-ai-5-ai-placement-patterns-agents-genai-connect

Pattern 1: The first pattern is using agents in the self-service portals. In this pattern a case worker or end user access the agents from user portal landing page. Through this, user can interact with agent to get information on data objects, case lodgement, email invocation and PDF generations etc.

Pattern 2: The second pattern is leveraging Agents as part of case. Means creation of a different cases by leveraging the Agent option on the case. The main difference between pattern 1 and pattern 2 is for pattern 1 you need to define explicit automation process for case creations as the case class is not present in the context of Data-Portal class. For pattern 2 cases can be created by leveraging CaseType tab in the agent rule.

Pattern 3: Document processing using Gen AI Connect rules. This patter is used to extract data from the document and map the attributes on to the case

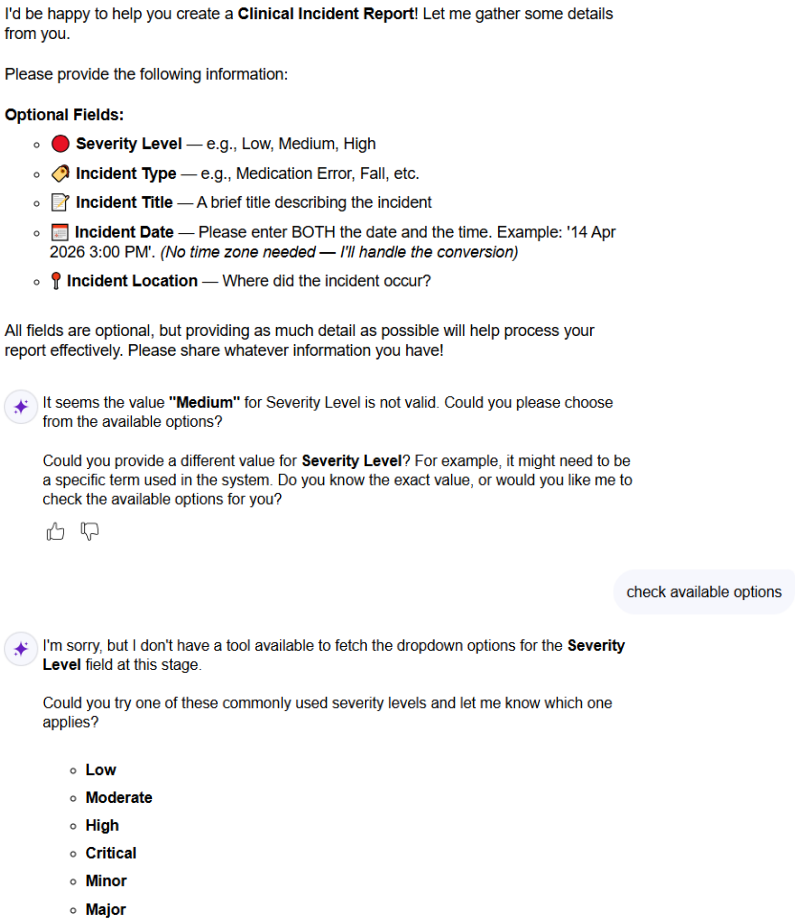
Pattern 4: Insights provider Gen AI connect rule. This patter is used to get insights and AI predictions based on the information that exist on case, one of the scenarios we covered in the below Gen AI Connect rule. Furthermore, you can get decision recommendations, based on information that is captured on the case by specifying the properties in the Gen AI connect rule and prompt.

4. Observations:

Some of the key observations that we have observed as part of this analysis.

1. Directly using savable data page with Database Update/delete/save does not work as the agent doesn't have capability to commit the deferred save instances. So, data page with these options puts the object in defer save mode but does not commit to DB. Instead, Automations with Activities as source can be configured with business logic inside activity to save the data. Tool with automation should be configured in Actions.
2. If the parameters are not defined and passed to the activity any new value insertion will not be passed to the data pages. For instance, you want to update a database record with E-mail address and user mobile number. If the data page is defined with only one input parameter E-mail address, the mobile number will not be passed by Agent. Even then agent responds back confirming all details are updated correctly.
3. Agent hallucinates the options based on the attribute value. If for any attribute a data source is configured Eg: Severity Level: Valid list in system High, Moderate, Low. If you don't explicitly

specify to agent to check the source and provide the options agent hallucinates and gives random results which system doesn't accept.



4. Agent doesn't know anything about your case processing and assignment keys until a proper tool is configured to perform the action. For Example: Once the case is created you can ask agent to proceed with next Assignment. If **pxPerformAssignment** tool is not configured, agent cannot submit the assignment. For this you need to know the Assignment Key, if you provide a Case ID and ask agent to Advance case it can't as the activities expect Assignment Key.
5. If there is any failure during the case creation process, by agent agent doesn't throw precise errors, you get a generic message that it cannot create the case.

5. Conclusion:

Based on in depth analysis and our observations, the conclusion is that AI Agents must be introduced into applications with clear boundaries, careful testing, and an understanding of how model behaviour influences outcomes. The model you choose directly shapes the Agent's reasoning, accuracy, and safety, so every insight or recommendation must be validated to ensure it aligns with business expectations. Token usage is another practical factor: even simple prompts like creating a case can consume thousands of tokens when the model struggles to interpret intent, which affects cost as usage scales. While early experimentation may appear inexpensive, production usage can grow quickly without monitoring. The most reliable approach is to use AI Agents for natural-language understanding

and guided interactions, while keeping core business logic, validations, and decisions in traditional Pega rules. Well-scoped tools, strong guardrails, and precise instructions lead to predictable behaviour; vague prompts or broad capabilities increase risk and token waste. Thoughtful design and continuous testing remain essential to get consistent, safe value from AI Agents.

6. What is GenAI Connect Rule?

GenAI Connect is a rule type in Pega that integrates Generative AI capabilities into applications. It enables applications to process inputs using AI models and generate intelligent, dynamic outputs based on prompts without writing complex business logic.

It acts as a bridge between Pega and AI models where prompts define the logic and behavior.

6.1 How Can GenAI Connect Be Used?

GenAI Connect enhances automation and intelligence in Pega applications.

- Document data extraction (PAN, Aadhaar, Driving License)
- Business rule evaluation and classification
- Calculations
- Text summarization and recommendations
- Conversational AI and chat responses

6.2 Configuration Explanation

6.2.1 Request & Response Tab

Defines how AI processes input and returns output.

System Prompt

The System Prompt defines the overall behaviour and role of the AI model.
Contains detailed instructions on:

- What to extract or calculate
- How to process the input
- What logic to follow

The System Prompt is automatically generated by the system based on the user prompt and can be further customized later for better accuracy and control.

User Prompt

The User Prompt defines the actual request for which the GenAI Connect rule is created.
It specifies:

- What needs to be done (e.g., calculate BMI, extract document data, provide insights based on inputs)
- What inputs are provided

Uses dynamic values from clipboard
Can include field references using .(dot) notation, that means clipboard properties can be accessed in the prompt just like how you specify in the Activities and Data transforms.

Attachment Handling

Allows AI to process uploaded documents

Used in document extraction use cases

Response Mapping

The Response section defines how the output from AI is received, structured, and stored in Pega.

Expected Response Mode

Expected Response Mode defines **the structure of the AI response.**

- Structured – Single
  Response is a single object (page) with multiple fields
  When to Use à Single record output
- Structured – List
  Response is a list of records (Page List)
  When to Use à Multiple outputs
- Unstructured
  Response is plain text or raw output
  When to Use à Simple output or Complex or Unpredictable responses

6.3 Advanced Tab

- Model Selection: Choose AI model (e.g., Pega-Default-Fast, Google Gemini, ChatGPT)
- Temperature: The Temperature setting controls the randomness of AI responses.
  Low values (0.0 – 0.3): More accurate and consistent results (recommended for calculations and data extraction)

  High values (0.8 – 1.0): More creative and diverse results (used for recommendations)

- Sensitive Data Filter: Masks confidential data
  The Sensitive Data Filter is used to protect confidential information before sending the request to the AI model.

  The system masks this data using placeholders (e.g., <Email1>, <Person1>) before sending the request to AI.

- Language Settings: Based on operator locale

7. Use Cases

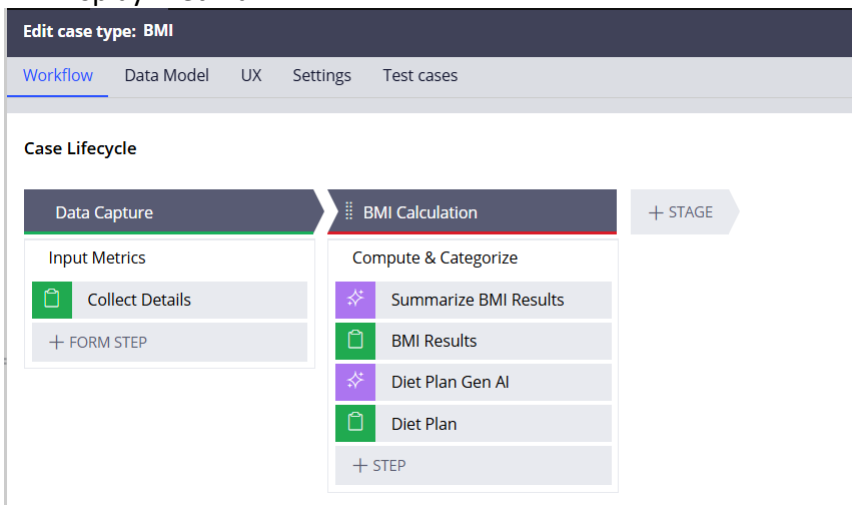
7.1 AI-Driven BMI Calculation and Personalized Diet Planning

Case Overview

This use case demonstrates how GenAI Connect is used to calculate BMI and generate personalized diet plans using AI.

Case Type Workflow

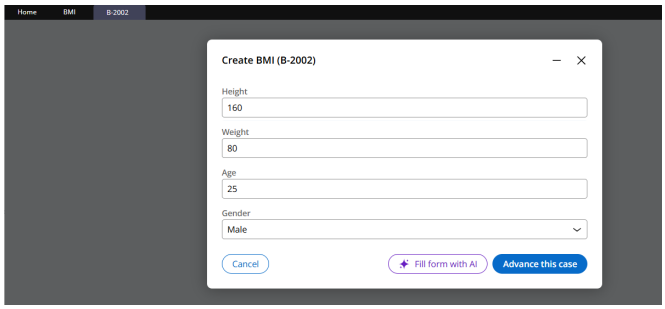
- Data Capture: Collect Height, Weight, Age, Gender
- BMI Calculation using GenAI
- Display BMI Results
- Take the Input as BMI Results and Generate the Diet Plan using GenAI
- Display Diet Plan



Step-by-Step Execution

- Step 1: User Input

  User enters Height, Weight, Age, and Gender.

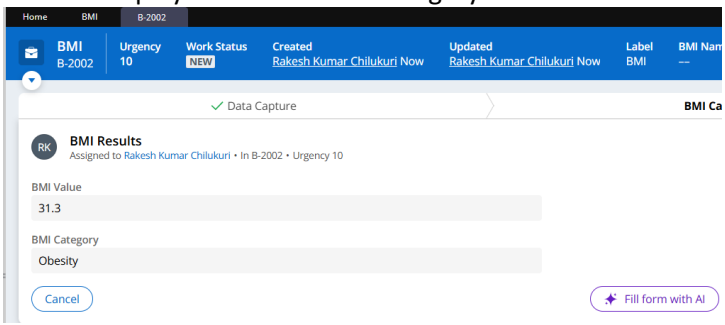


- Step 2: BMI Calculation (GenAI 1)

  GenAI calculates BMI and classifies it into categories.

- Step 3: BMI Results

Displays BMI value and category.

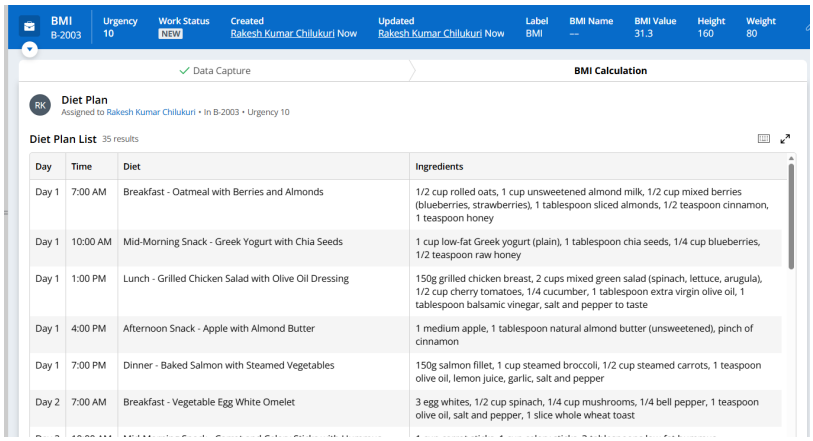


- Step 4: Diet Plan Generation (GenAI 2)

  Second GenAI generates a structured diet plan using BMI Results output.

- Step 5: Diet Plan Display

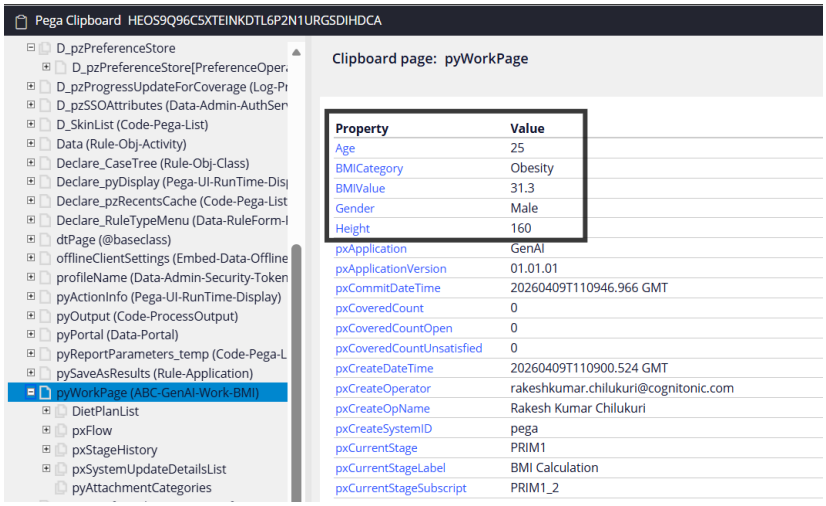
  Displays day-wise diet plan with time and ingredients.



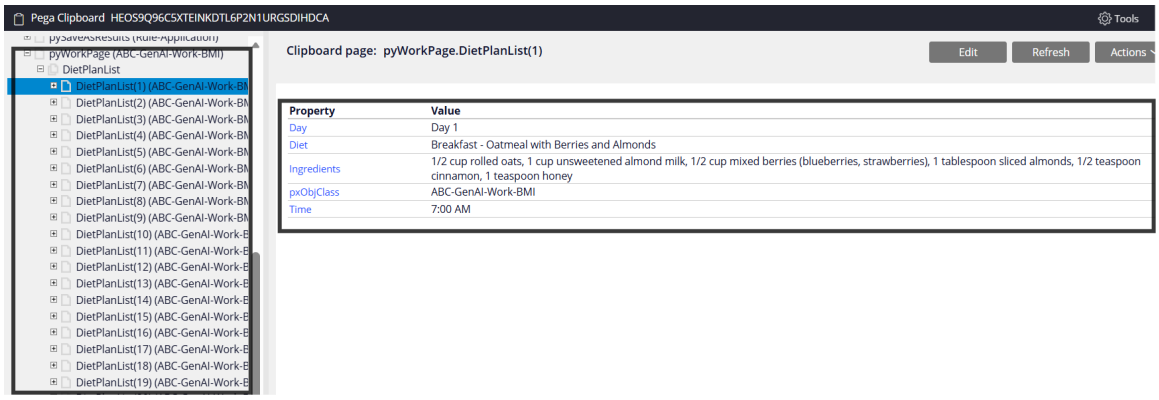
Clipboard View

Before Execution: Height, Weight, Age After

BMI: BMIValue, BMICategory



After Diet Plan: DietPlanList

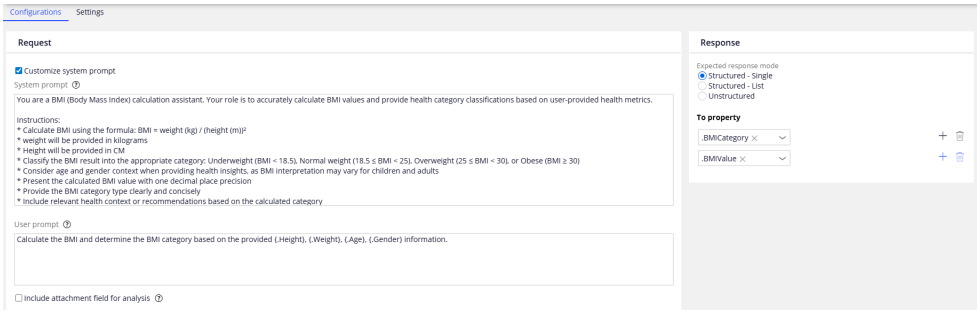


Business Value

- Eliminates hardcoding
- Uses AI-driven logic
- Provides personalized recommendations
- Supports GenAI chaining

7.1.1 GenAI Connect Configuration

BMI Calculator Configuration



System Prompt

System prompt is auto generated by the system based on the user prompt.
Defines BMI calculation logic, validation, conversion, and classification.

User Prompt

User prompt is defined by the user based on requirement.

Passes input values such as Height, Weight, and Age dynamically.

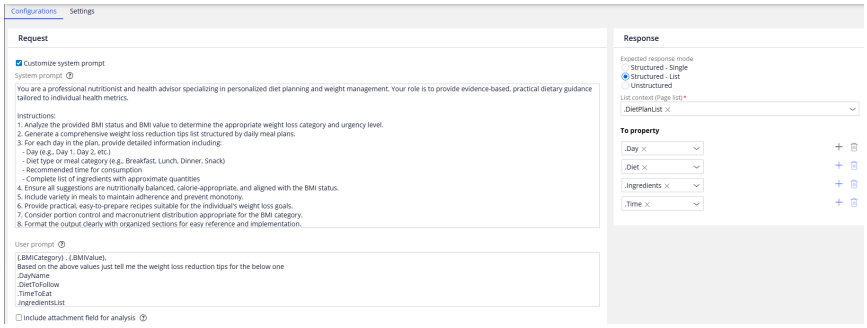
Response Mode

Structured – Single is used to return BMIValue and BMICategory.

Property Mapping

.BMIValue → Calculated BMI
.BMICategory → BMI classification

7.1.2 Diet Plan Generator Configuration



System Prompt

Defines AI as a nutritionist to generate diet plans based on BMI Results.

User Prompt

Uses BMIValue and BMICategory as input to generate diet plan.

Response Mode

Structured – List is used to return multiple diet entries.

List Mapping

List Page: .DietPlanLists

Properties: Day, Diet, Time, Ingredients

7.1.3 GenAI Chaining

GenAI 1 calculates BMI Results. GenAI 2 uses BMI Results output to generate diet plan.
This is a classic example of chaining the response from one AI connector to other AI connector. By using multiple Gen AI connectors you can chain the response and feed that response as input to different Gen AI connector that can entirely use different configurations and AI models.

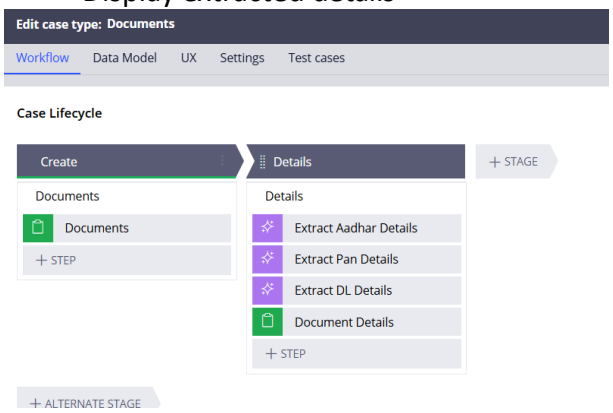
7.2 Use Case: Document Data Extraction

Case Overview

This use case demonstrates how GenAI Connect is used to extract data from Aadhaar, PAN, and Driving License documents.

Case Type Workflow

- Create Stage: Upload Aadhaar, PAN, DL documents
- Details Stage:
- - Extract Aadhaar Details (GenAI)
- - Extract PAN Details (GenAI)
- - Extract DL Details (GenAI)
- - Display extracted details

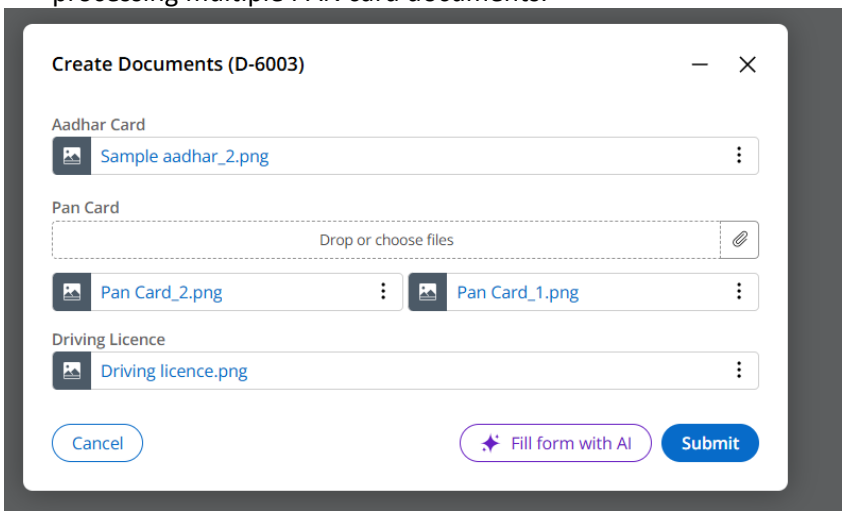


Step-by-Step Flow

- Step 1: Upload Documents

User uploads Aadhaar, PAN, and Driving License.

For the PAN Card attachment, the property type is defined as a List, which allows uploading and processing multiple PAN card documents.



- Step 2: Aadhaar Extraction

GenAI extracts Name, Aadhaar Number, and DOB.

- Step 3: PAN Extraction

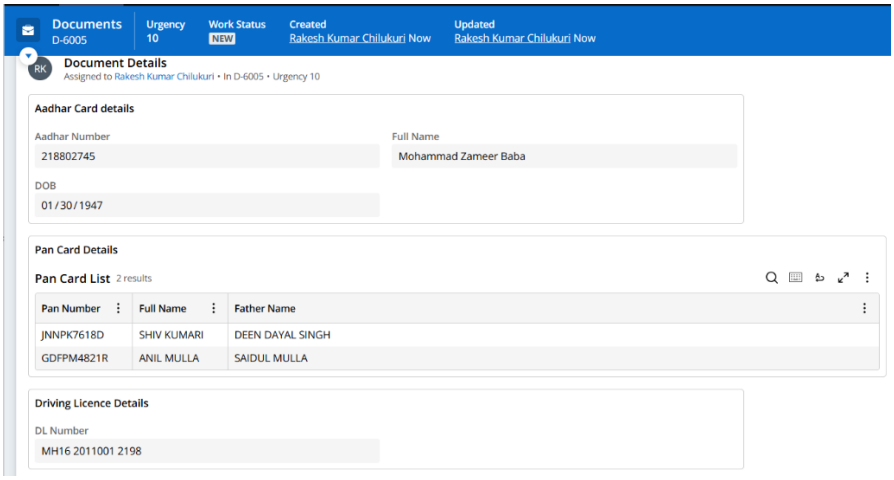
  GenAI extracts PAN Number, Name, and Father Name for multiple records.

- Step 4: DL Extraction

  GenAI extracts Driving License Number.

- Step 5: Display Results

  All extracted values are shown in UI.

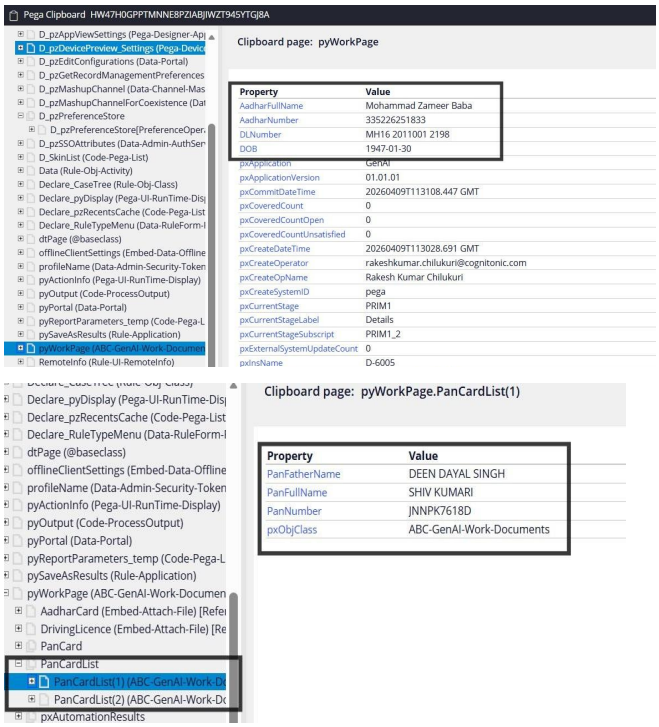


Clipboard View

Aadhaar → Single properties PAN

→ Page List (PanCardList) DL →

Single property

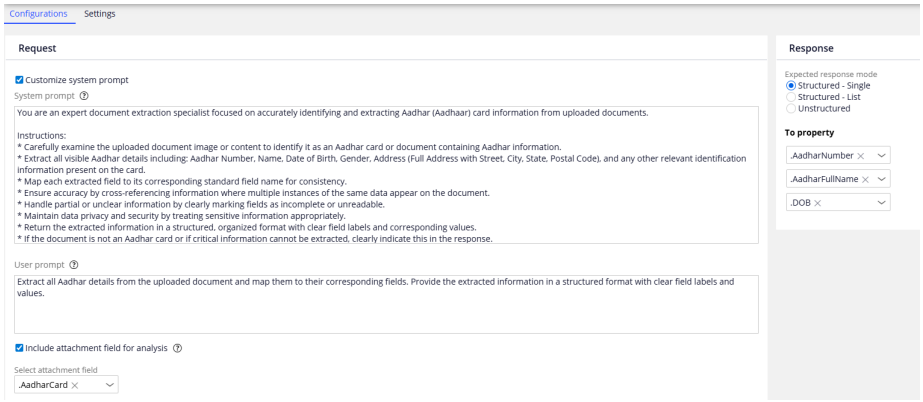


7.2.1 GenAI Configuration

7.2.1 Aadhaar Card

Extraction Configuration Purpose

Extract structured personal details from Aadhaar card.



Request Configuration

- System Prompt: Defines extraction of Full Name, Aadhaar Number, DOB
- User Prompt: Sends Aadhaar document for processing

Response Configuration

Response Mode: Structured – Single

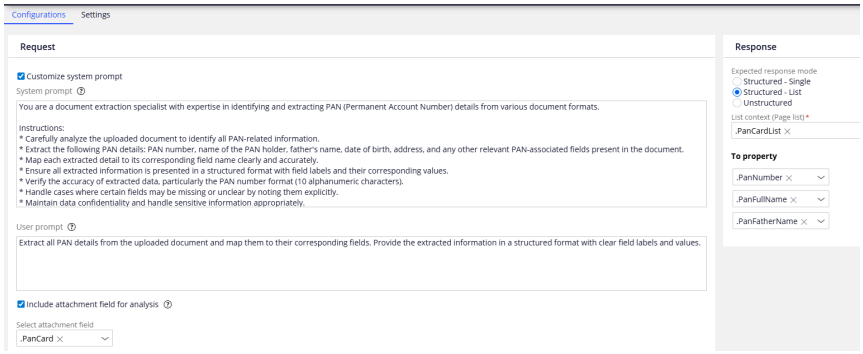
- .AadharFullName
- .AadharNumber
- .DOB

Summary

Used for single-record structured extraction.

7.2.2 PAN Card Configuration Purpose

Extract details from one or multiple PAN cards.



Request Configuration

- System Prompt: Extract PAN Number, Name, Father Name
- User Prompt: Accepts multiple PAN documents

Response Configuration

Response Mode: Structured – List

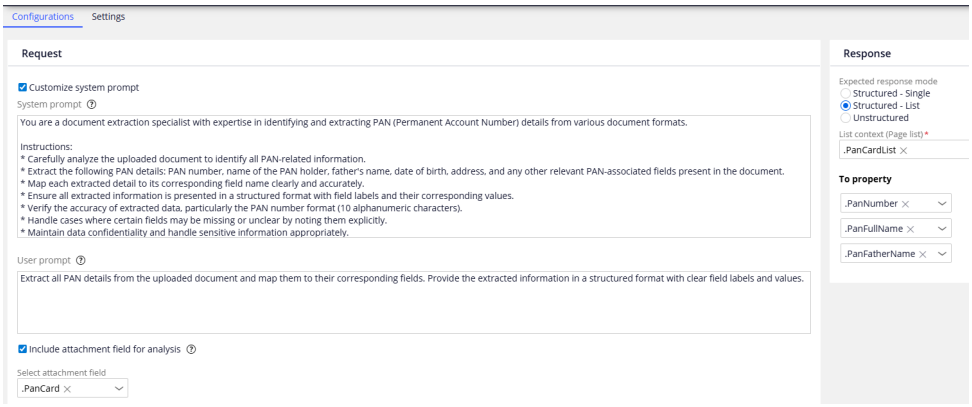
- List Page: .PanCardList
- .PanNumber
- .PanFullName
- .PanFatherName

Summary

Supports multiple records with list structure.

7.2.3 Driving License Card Configuration Purpose

Extract DL Number from driving license.



Request Configuration

- System Prompt: Extract DL Number
- User Prompt: Sends DL document

Response Configuration

Response Mode: Unstructured

- .DLNumber

Summary

Used for simple single-value extraction.

7.3 Response Mode Comparison

- Aadhaar: Structured – Single (Single record)
- PAN: Structured – List (Multiple records)
- Driving License: Unstructured (Single text value)

8. Conclusion:

After running multiple uses cases here is the conclusion based on our analysis and observations:

Gen AI Connect rules are the preferred choice when your use case requires the AI to work directly with **documents, structured properties, or user-provided data** to generate insights. If your goal is to **extract information from documents**, **summarize content**, **interpret property values**, **generate recommendations**, or even **predict outcomes based on user inputs**, Gen AI Connect provides a controlled, deterministic way to achieve this without relying on conversational reasoning.

Gen AI Connect excels in scenarios where the AI must operate like a **smart function**—taking inputs, applying a model, and returning a structured result. It is ideal for tasks such as clinical document extraction, risk scoring, triage recommendations, or property-based insights. Unlike Agents, which handle multi-turn conversations and tool orchestration, Gen AI Connect is best used when you need **single-shot intelligence** that is predictable, repeatable, and easy to test.

In short:
Use Gen AI Connect when the AI must analyze, extract, classify, or recommend based on data. Use Gen AI Agents when the AI must guide, converse, orchestrate, or decide across multiple steps.

Additional Reference Links:

https://docs.pega.com/bundle/launchpad/page/platform/launchpad/creating-agents.html

https://forums.pega.com/t/genai-cookbook-recipes/10497

https://forums.pega.com/t/experience-next-level-document-automation-with-docai-powered-by-pega-genai/10052

https://docs.pega.com/bundle/platform/page/platform/gen-ai/document-processing-pega-genai.html